Tarea M24-CD Javier Bueno Sanchez

July 14, 2026

1 Tarea M24-CD Javier Bueno Sanchez.

Preámbulo:

En la actividad anterior se desarrolló un árbol de decisión capaz de clasificar correctamente la mayoría de los pacientes según el medicamento al que respondieron. En esta práctica se evaluará si diferentes métodos de ensamble permiten mejorar ese rendimiento predictivo o si el árbol individual ya representa una solución suficientemente robusta para este conjunto de datos.

Índice:

1. Objetivo.

2. Variable objetivo.

3. Importación de librerías.

4. Carga del conjunto de datos.

5. Análisis exploratorio de los datos.

6. Preparación de datos.

7. Construcción del modelo.

8. Evaluación.

9. Interpretación.

10. Conclusiones.

1. Objetivo:

   Efectuar un análisis de clasificación mediante ensambles que permita desarrollar un modelo predictivo basado en distintas métricas aplicadas a una base de datos grande.

Problema: Imagina que eres un investigador médico que recopila datos para un estudio. Has recopilado datos sobre un conjunto de pacientes, todos ellos con la misma enfermedad. Durante su tratamiento, cada paciente ha respondido a uno de los 5 medicamentos, el fármaco A, el fármaco B, el fármaco c, el fármaco X y el Y.

Parte de tu trabajo consiste en construir un modelo para averiguar qué medicamento podría ser apropiado para un futuro paciente con la misma enfermedad. Los conjuntos de características de este conjunto de datos son la edad, el sexo, la presión arterial y el colesterol de los pacientes, y el objetivo es estudiar el fármaco al que respondió cada paciente.

Se trata de un ejemplo de clasificación multiclase, y se puede utilizar la parte de entrenamiento del conjunto de datos para construir un árbol de decisión, y luego utilizarlo para predecir la clase de un paciente desconocido, o para prescribirlo a un nuevo paciente.

2. Variable objetivo.

[40]:
```
from IPython.display import Image, display

display(Image(filename='/home/javi/Imágenes/variables.png'))
```

Age	Sex	BP	Cholesterol	Na_to_K	Drug
23	F	HIGH	HIGH	25.355	drugY
47	M	LOW	HIGH	13.093	drugC
47	M	LOW	HIGH	10.114	drugC
28	F	NORMAL	HIGH	7.798	drugX
61	F	LOW	HIGH	18.043	drugY
22	F	NORMAL	HIGH	8.607	drugX
49	F	NORMAL	HIGH	16.275	drugY
41	M	LOW	HIGH	11.037	drugC

Paso a paso:

Cargue la base de datos “drugs.csv” en Python e investigue cómo convertir las variables predictoras cualitativas de esta base a una escala numérica mediante la instrucción “preprocessing.LabelEncoder()”. Por ejemplo, si una variable tiene 3 posibles categorías, deberá cambiar sus resultados a 0, 1 o 2.

Use los diversos métodos de ensambles vistos en este módulo para encontrar un algoritmo óptimo de clasificación. Explique cuál sería su recomendación para este caso.

¿Mejoró su poder predictivo al comparar sus resultados con los obtenidos mediante el árbol de decisión aplicado en la actividad del módulo previo? Explique.

3. Importación de librerías.

[41]:
```
# Manipulación de datos
import pandas as pd
import numpy as np

# Visualización
import matplotlib.pyplot as plt

# División y validación de datos
from sklearn.model_selection import (train_test_split, StratifiedKFold,␣
 ↪cross_validate)

# Modelos de clasificación
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (RandomForestClassifier,␣
 ↪AdaBoostClassifier,GradientBoostingClassifier)

# Métricas de evaluación
from sklearn.metrics import (accuracy_score, precision_score, recall_score,␣
 ↪f1_score,
    classification_report, confusion_matrix, ConfusionMatrixDisplay)

# Codificación de variables categóricas
from sklearn.preprocessing import LabelEncoder

# Advertencias
import warnings
warnings.filterwarnings("ignore")
```

[42]:
```
#Verificamos que todas las librerias quedaron instaladas y su version.
import pandas
import numpy
import sklearn
import matplotlib

print("Pandas:", pd.__version__)
print("NumPy:", numpy.__version__)
print("Scikit-Learn:", sklearn.__version__)
print("Matplotlib:", matplotlib.__version__)
```

```
Pandas: 2.3.3
NumPy: 2.3.5
Scikit-Learn: 1.7.2
Matplotlib: 3.10.6
```

4. Carga del conjunto de datos.

[43]:
```
# Cargamos el archivo
df = pd.read_csv("/home/javi/Descargas/drugs(1).csv")
```

5. Análisis exploratorio de los datos.

[44]:
```
# Visualizamos archivo.
df.head()
```

[44]:

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	F	HIGH	HIGH	25.355	drugY
1	47	M	LOW	HIGH	13.093	drugC
2	47	M	LOW	HIGH	10.114	drugC
3	28	F	NORMAL	HIGH	7.798	drugX
4	61	F	LOW	HIGH	18.043	drugY

[45]:
```
# Dimensiones del conjunto de datos.
print(f"Filas: {df.shape[0]}")
print(f"Columnas: {df.shape[1]}")
```

```
Filas: 200
Columnas: 6
```

[46]:
```
# Nos muestra valores.
df.describe()
```

[46]:

	Age	Na_to_K
count	200.000000	200.000000
mean	44.315000	16.084485
std	16.544315	7.223956
min	15.000000	6.269000
25%	31.000000	10.445500
50%	45.000000	13.936500
75%	58.000000	19.380000
max	74.000000	38.247000

[47]:
```
# Valores nulos por variable.
df.isnull().sum()
```

[47]:
```
Age            0
Sex            0
BP             0
Cholesterol    0
Na_to_K        0
Drug           0
dtype: int64
```

[48]:
```
# Información general del dataset.
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 6 columns):
```

```
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Age          200 non-null    int64
 1   Sex          200 non-null    object
 2   BP           200 non-null    object
 3   Cholesterol  200 non-null    object
 4   Na_to_K      200 non-null    float64
 5   Drug         200 non-null    object
dtypes: float64(1), int64(1), object(4)
memory usage: 9.5+ KB
```

[49]:
```
# Verificamos que no tengamos datos duplicados.
df.duplicated().sum()
```

[49]:
```
np.int64(0)
```

6. Preparación de datos.

Dado que el modelo utiliza árboles de decisión y métodos de ensamble basados en árboles, es necesario convertir las variables categóricas a una representación numérica mediante LabelEncoder. Esta transformación permite que los algoritmos procesen correctamente variables como el sexo, la presión arterial, el colesterol y el medicamento objetivo, manteniendo una correspondencia entre cada categoría y un valor numérico.

[50]:
```
from sklearn.preprocessing import LabelEncoder

le_sex = LabelEncoder()
le_bp = LabelEncoder()
le_chol = LabelEncoder()
le_drug = LabelEncoder()

df["Sex"] = le_sex.fit_transform(df["Sex"])
df["BP"] = le_bp.fit_transform(df["BP"])
df["Cholesterol"] = le_chol.fit_transform(df["Cholesterol"])
df["Drug"] = le_drug.fit_transform(df["Drug"])

print("Sex:", dict(zip(le_sex.classes_, le_sex.transform(le_sex.classes_))))
print("BP:", dict(zip(le_bp.classes_, le_bp.transform(le_bp.classes_))))
print("Cholesterol:", dict(zip(le_chol.classes_, le_chol.transform(le_chol.
  ↪classes_))))
print("Drug:", dict(zip(le_drug.classes_, le_drug.transform(le_drug.classes_))))
```

```
Sex: {'F': np.int64(0), 'M': np.int64(1)}
BP: {'HIGH': np.int64(0), 'LOW': np.int64(1), 'NORMAL': np.int64(2)}
Cholesterol: {'HIGH': np.int64(0), 'NORMAL': np.int64(1)}
Drug: {'drugA': np.int64(0), 'drugB': np.int64(1), 'drugC': np.int64(2),
'drugX': np.int64(3), 'drugY': np.int64(4)}
```

Las variables categóricas fueron transformadas a valores numéricos mediante LabelEncoder, permitiendo que el algoritmo de Árboles de Decisión pueda procesarlas. La correspondencia entre

categorías y valores se conserva para interpretar correctamente las predicciones del modelo.

7. Construcción del modelo.

[51]:
```
# Variables predictoras
X = df.drop("Drug", axis=1)

# Variable objetivo
y = df["Drug"]

print("Dimensiones de X:", X.shape)
print("Dimensiones de y:", y.shape)
```

```
Dimensiones de X: (200, 5)
Dimensiones de y: (200,)
```

[52]:
```
# División de datos.
from sklearn.model_selection import train_test_split

# División del conjunto de datos en entrenamiento y prueba.
# test_size = 30% de los datos se guardan para prueba.
# random_state = 42 asegura que el resultado sea reproducible.
# stratify = y mantiene la misma proporción de las clases en los conjuntos de␣
 ↪entrenamiento y prueba.
# De esta forma, ambos representan adecuadamente la distribución original de la␣
 ↪base de datos y se evita
# que una división aleatoria genere muestras poco representativas, lo que␣
 ↪podría afectar la evaluación de modelo.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30,␣
 ↪random_state=42, stratify=y)

print("Entrenamiento:", X_train.shape)
print("Prueba:", X_test.shape)
print("Entrenamiento:", y_train.shape)
print("Prueba:", y_test.shape)
```

```
Entrenamiento: (140, 5)
Prueba: (60, 5)
Entrenamiento: (140,)
Prueba: (60,)
```

Imprimimos resultados para verificar que: - Ningún registro se perdió. - La variable objetivo mantiene correspondencia con las variables predictoras. - La división es idéntica a la del módulo anterior. - La comparación entre el árbol de decisión y los métodos de ensamble será justa.

Iniciamos Arbol de Decision.

[53]:
```
from sklearn.tree import DecisionTreeClassifier

# Crear el modelo.
```

```
modelo_arbol = DecisionTreeClassifier(criterion="entropy", max_depth=4,␣
 ↪random_state=42)

# Entrenar.
modelo_arbol.fit(X_train, y_train)
```

[53]:
```
DecisionTreeClassifier(criterion='entropy', max_depth=4, random_state=42)
```

[54]:
```
# Generamos las predicciones.
y_pred_arbol = modelo_arbol.predict(X_test)
```

[55]:
```
# Evaluar el modelo.
from sklearn.metrics import accuracy_score, classification_report,␣
 ↪confusion_matrix

print(f"Accuracy: {accuracy_score(y_test, y_pred_arbol):.4f}")

print("\nMatriz de confusión")
print(confusion_matrix(y_test, y_pred_arbol))

print("\nReporte de clasificación")
print(classification_report(y_test, y_pred_arbol))
```

```
Accuracy: 0.9833

Matriz de confusión
[[ 7  0  0  0  0]
 [ 1  4  0  0  0]
 [ 0  0  5  0  0]
 [ 0  0  0 16  0]
 [ 0  0  0  0 27]]

Reporte de clasificación
              precision    recall  f1-score   support

           0       0.88      1.00      0.93         7
           1       1.00      0.80      0.89         5
           2       1.00      1.00      1.00         5
           3       1.00      1.00      1.00        16
           4       1.00      1.00      1.00        27

    accuracy                           0.98        60
   macro avg       0.97      0.96      0.96        60
weighted avg       0.99      0.98      0.98        60
```

Podemos observar que el dato Accuracy que nos arroja es igual al de la práctica anterior, lo que nos ayuda a confirmar que la base de datos es la misma, que la codificación es la correcta, que

entrenamiento y prueba es la misma y el modelo se reprodujo correctamente.

En la Matriz de confusión tuvo un error donde un paciente con drugB lo clasificó como drugA, que nos lleva a ver que la Precisión de 0.88 es porque indicó que 8 pacientes son drugA pero en realidad solo son 7, pero un Recall de 1.00 nos indica que todos son drugA (verdaderos), acertó todos. Si observamos drugB tenemos el opuesto, precisión de 1.00 acertando cuando era drugb, pero Recall de 0.88 viene que de 5 verdaderos encontró a 4.

Que tengamos un Macro F1 de 0.96 nos permite identificar que todas las clases tienen el mismo peso, aunque drugY con 27 pacientes tenga una mayor dominancia de resultado.

Iniciamos Random Forest.

[56]:
```
# Creamos el modelo Random Forest.
modelo_rf = RandomForestClassifier(random_state=42)

# Entrenar.
modelo_rf.fit(X_train, y_train)
```

[56]:
```
RandomForestClassifier(random_state=42)
```

[57]:
```
# Generamos las predicciones.
y_pred_arbol = modelo_rf.predict(X_test)
```

[58]:
```
# Evaluar el modelo.
from sklearn.metrics import accuracy_score, classification_report,␣
 ↪confusion_matrix

print(f"Accuracy: {accuracy_score(y_test, y_pred_arbol):.4f}")

print("\nMatriz de confusión")
print(confusion_matrix(y_test, y_pred_arbol))

print("\nReporte de clasificación")
print(classification_report(y_test, y_pred_arbol))
```

```
Accuracy: 0.9833

Matriz de confusión
[[ 7  0  0  0  0]
 [ 1  4  0  0  0]
 [ 0  0  5  0  0]
 [ 0  0  0 16  0]
 [ 0  0  0  0 27]]

Reporte de clasificación
              precision    recall  f1-score   support

           0       0.88      1.00      0.93         7
           1       1.00      0.80      0.89         5
```

```
           2       1.00      1.00      1.00         5
           3       1.00      1.00      1.00        16
           4       1.00      1.00      1.00        27

    accuracy                           0.98        60
   macro avg       0.97      0.96      0.96        60
weighted avg       0.99      0.98      0.98        60
```

Random Forest vino a confirmar los resultados obtenidos con el árbol de decisión, sin evidenciar una mejora en el poder predictivo para este conjunto de datos.

Iniciamos Ada Boost Classified.

[60]:
```
from sklearn.ensemble import (RandomForestClassifier,␣
 ↪AdaBoostClassifier,GradientBoostingClassifier)

# Crear el modelo
modelo_ada = AdaBoostClassifier(random_state=42)

# Entrenar
modelo_ada.fit(X_train, y_train)
```

[60]:
```
AdaBoostClassifier(random_state=42)
```

[63]:
```
# Generamos las predicciones.
y_pred_arbol = modelo_ada.predict(X_test)
```

[64]:
```
# Evaluar el modelo.
from sklearn.metrics import accuracy_score, classification_report,␣
 ↪confusion_matrix

print(f"Accuracy: {accuracy_score(y_test, y_pred_arbol):.4f}")

print("\nMatriz de confusión")
print(confusion_matrix(y_test, y_pred_arbol))

print("\nReporte de clasificación")
print(classification_report(y_test, y_pred_arbol))
```

```
Accuracy: 0.8333

Matriz de confusión
[[ 7  0  0  0  0]
 [ 5  0  0  0  0]
 [ 0  0  0  5  0]
 [ 0  0  0 16  0]
 [ 0  0  0  0 27]]
```

```
Reporte de clasificación
              precision    recall  f1-score   support

           0       0.58      1.00      0.74         7
           1       0.00      0.00      0.00         5
           2       0.00      0.00      0.00         5
           3       0.76      1.00      0.86        16
           4       1.00      1.00      1.00        27

    accuracy                           0.83        60
   macro avg       0.47      0.60      0.52        60
weighted avg       0.72      0.83      0.77        60
```

Para este conjunto de datos y con la configuración utilizada, AdaBoost obtuvo un rendimiento inferior al árbol de decisión y a Random Forest.

AdaBoost concentró sus errores en dos clases específicas (drugB y drugC), lo que indica que, para este conjunto de datos y con la configuración utilizada, no consiguió separar adecuadamente esas categorías. En consecuencia, su rendimiento fue inferior al del árbol de decisión y Random Forest.

Inciamos Gradient Boosting Classifier.

[69]:
```
from sklearn.ensemble import (RandomForestClassifier,␣
 ↪AdaBoostClassifier,GradientBoostingClassifier)

# Crear el modelo
modelo_gra = GradientBoostingClassifier(random_state=42)

# Entrenar
modelo_gra.fit(X_train, y_train)
```

[69]:
```
GradientBoostingClassifier(random_state=42)
```

[70]:
```
# Generamos las predicciones.
y_pred_arbol = modelo_gra.predict(X_test)
```

[71]:
```
# Evaluar el modelo.
from sklearn.metrics import accuracy_score, classification_report,␣
 ↪confusion_matrix

print(f"Accuracy: {accuracy_score(y_test, y_pred_arbol):.4f}")

print("\nMatriz de confusión")
print(confusion_matrix(y_test, y_pred_arbol))

print("\nReporte de clasificación")
print(classification_report(y_test, y_pred_arbol))
```

```
Accuracy: 0.9667
```

```
Matriz de confusión
[[ 7  0  0  0  0]
 [ 1  3  0  1  0]
 [ 0  0  5  0  0]
 [ 0  0  0 16  0]
 [ 0  0  0  0 27]]

Reporte de clasificación
              precision    recall  f1-score   support

           0       0.88      1.00      0.93         7
           1       1.00      0.60      0.75         5
           2       1.00      1.00      1.00         5
           3       0.94      1.00      0.97        16
           4       1.00      1.00      1.00        27

    accuracy                           0.97        60
   macro avg       0.96      0.92      0.93        60
weighted avg       0.97      0.97      0.96        60
```

Los resultados están contando una historia muy coherente. Gradient Boosting solo se equivoca en la clase drugB, ya no falla con drugC, como ocurrió con AdaBoost.

Esto confirma que Gradient Boosting sigue siendo un modelo sólido, pero no alcanza el rendimiento del árbol de decisión ni de Random Forest en este conjunto de datos.

[79]:
```
# Tabla comparativa de modelos
comparacion = pd.DataFrame({"Modelo": ["Árbol de Decisión", "Random Forest",␣
 ↪"AdaBoost","Gradient Boosting"],
    "Accuracy": [0.9833, 0.9833, 0.8333, 0.9667],
    "Macro F1": [0.96, 0.96, 0.52, 0.93],
    "Errores": [1, 1, 10, 2],
    "Observaciones": ["Modelo base con excelente desempeño.", "Confirma los␣
 ↪resultados del árbol.",
          "No distingue correctamente dos clases.","Buen desempeño, pero␣
 ↪introduce un error adicional."]})

# Le damos ajustes visuales a la tabla para una mejor lectura.
comparacion["Accuracy"] = comparacion["Accuracy"].map("{:.2%}".format)
comparacion["Macro F1"] = comparacion["Macro F1"].map("{:.2f}".format)
comparacion.style.highlight_max(subset=["Accuracy"], color="lightgreen")

comparacion
```

[79]:

	Modelo	Accuracy	Macro F1	Errores	\
0	Árbol de Decisión	98.33%	0.96	1	

```
1      Random Forest   98.33%     0.96        1
2           AdaBoost   83.33%     0.52       10
3  Gradient Boosting   96.67%     0.93        2

                                      Observaciones
0               Modelo base con excelente desempeño.
1                 Confirma los resultados del árbol.
2             No distingue correctamente dos clases.
3  Buen desempeño, pero introduce un error adicio…
```

Las métricas permiten comparar el rendimiento global de los modelos; la matriz de confusión explica el origen de las diferencias observadas.

8. Evaluación.

Los resultados obtenidos muestran un comportamiento consistente entre los modelos evaluados, lo que aumenta la confianza en que el conjunto de datos es adecuado para este problema de clasificación. De los cuatro modelos, Árbol de Decisión y Random Forest obtienen excelente desempeño con una precisión del 98.33%, soportando sus resultados mutuamente, por otro lado tenemos Gradient Boosting que presentó un desempeño cercano, aunque con un ligero incremento en los errores de clasificación, AdaBoost por el contrario redujo su precisión e introduciendo más errores.

La recomendación sobre cual modelo aplicar sería tanto Árbol de Decisión como Random Forest, puesto que no se observan diferencias significativas.

9. Interpretación.

¿Mejoró su poder predictivo al comparar sus resultados con los obtenidos mediante el árbol de decisión aplicado en la actividad del módulo previo?

Se igualó el poder predictivo del Árbol de Decisión de la anterior práctica, lo que indica que el árbol de decisión ya capturaba adecuadamente los patrones presentes en los datos y que los métodos de ensamble no aportaron una mejora en el poder predictivo para este conjunto de datos.

10. Conclusiones.

- Se implementó correctamente un modelo de Árbol de Decisión y los métodos de ensambles RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier utilizando Scikit-learn para clasificar el medicamento recomendado a partir de las características clínicas de los pacientes. Los métodos de ensambles RandomForestClassifier obtuvo el mismo desempeño que el Árbol de Decisión con una exactitud del 98.33%, mientras que AdaBoostClassifier y GradientBoostingClassifier obtuvieron el mismo desempeños menores sobre el conjunto de prueba, alcanzando una exactitud del 83.33% y 96.67% respectivamente, lo que indica que Árbol de Decisión y RandomForestClassifier presentaron un rendimiento equivalente en este conjunto de datos.

[ ]: